

# 基於大語言模型和思維鏈推理的程式碼審查框架

## A Code Review Framework based on Large Language Models and Chain-of-Thought Reasoning

陳冠穎 李文廷

Jeffrey Chen, Wen-Tin Lee

Department of Software Engineering and Management

National Kaohsiung Normal University

611377102@mail.nknu.edu.tw, wtlee@mail.nknu.edu.tw

### 摘要

人工程式碼審查雖能確保品質與一致性，卻高度依賴審查者經驗，於快速迭代環境難以兼顧效率與一致性。直接把大型語言模型（Large Language Models, LLMs）用於審查，又會遇到三個關卡：模型易產生幻覺（編造看似合理卻不正確之內容）、同一份程式碼前後意見不一致、且不認得個別專案之規範。為同時回應此三者，本研究提出一套整合框架：以思維鏈（Chain-of-Thought, CoT）推理把審查拆成摘要、初步審查、靜態分析、程式碼異味偵測與總結等五個循序步驟，讓每步只證一件事而可逐步追溯（對應一致性），以檢索增強生成（Retrieval-Augmented Generation, RAG）在推理前動態注入專案規範以抑制幻覺（對應可解釋性），再以知識蒸餾（Knowledge Distillation, KD）結合量化低秩適應（Quantized Low-Rank Adaptation, QLoRA）把教師之審查能力轉移至可於有限資源部署之輕量學生模型（對應可落地之可維護性）。於 CRSCORE++ 指標上，同一 judge、標準評分提示詞下本框架以 gemma-4-31B-it 與前代學生模型為基底之品質相當（見表一），且高於基準。消融分析顯示多階段提示詞為單項貢獻最大之一環（+34 分），LoRA 微調之邊際貢獻雖較小（+2 分），卻解決有限資源下保留並穩定教師審查能力此一不同問題，二者互補。唯有三項技術整合，方便審查同時具備一致性、可解釋性與可維護性。

**關鍵字：**程式碼審查、大型語言模型、思維鏈推理、檢索增強生成、知識蒸餾、LLM-as-a-Judge、軟體工程自動化

### 一、引言

#### 1.1 研究背景

大型語言模型於自然語言與程式語言理解上的突破，正快速滲透軟體工程各環節，從程式碼生成、缺陷偵測到文件撰寫均見其應用。程式碼審查（Code Review）係指開發者相互檢視彼此之程式碼變更，不僅確保品質、可維護性與一致性，更是團隊協作與知識傳遞之核心機制，使潛在邏輯錯誤、安全漏洞與效能問題得於整合測試前被發現。然而隨專案規模與迭代速度上升，純人工審查之時間與人力成本日增，逐漸成為制約開發效率與審查一致性之瓶頸，促使研究社群以人工智慧提升審查之自動化與可靠性。

#### 1.2 研究動機

人工審查的侷限可歸納為三點。其一，處理審查意見所需之時間隨意見數量線性增加 [4]，於大型複雜專案中尤易形成瓶頸 [20]，並對資深工程師造成額外負擔 [5]。其二，傳統靜態分析（Static Analysis）與 Lint 工具雖能於執行前依規則檢出語法與風格問題，卻受限於規則僵化，難以涵蓋設計合理性、命名語義等高層次判斷 [1][6][12]。其三，將大型語言模型導入審查雖能跨越自然語言與程式結構之鴻溝、掌握語義與設計意圖，但模型本身帶來三個彼此關聯之障礙：易產生幻覺、對同一份程式碼前後意見不一致、且缺乏對個別專案領域規範之約束，往往需搭配人工監督 [15]。此三者正對應審查最需要的可解釋性、一致性與可維護性，如何在發揮 LLM 語義理解能力之同時約束其輸出、使審查兼具此三性質，即為本研究之核心動機，亦是 §1.3 與 §1.4 逐一對應之起點。

#### 1.3 研究目的與貢獻

針對上述三問題，本研究的構想是：與其分別修補幻覺、不穩定與缺領域規範，不如把彼此互補的三項技術整合成一套框架，讓它們各自負責一個問題又能協同運作。框架以多階段分析將審查拆解為摘要生成、初步審查、靜態分析、程式碼異味偵測與總結報告等循序步驟（思維鏈 CoT 係一種引導模型先逐步推理、再給出結論之提示策略），使每步僅承擔單一子任務、產物逐一落檔而可追溯，以換取一致性。RAG 則將專案規範與領域知識在推理前注入脈絡，於不重新訓練模型之前提下抑制幻覺，以換取可解釋性。模型面採知識蒸餾結合量化低秩適應（Quantized Low-Rank Adaptation, QLoRA），將教師之高階推理能力轉移至輕量化學生模型（現行基底為 gemma-4-31B-it, Google 釋出之開放權重大型語言模型），以換取可落地之可維護性。

據此，本研究之貢獻可歸納為四項。其一，多階段 CoT 審查流程，將審查結構化為可逐步推理且可追溯之循序步驟。其二，以知識蒸餾結合 QLoRA 達成輕量化部署，使高品質審查能力於有限運算資源下落地。其三，以 RAG 動態注入專案規則，使審查貼合個別專案之領域規範。其四，提出 LLM-as-a-Judge-Our 五維評估指標，以細化審查品質之量測。此四者並非各自獨立，而是相互補位：唯有整合，審查方能同時具備一致性、可解釋性與可維護性，正面回應前述三大問題，此亦本框架有別於既有單一面向方法之處。實驗

顯示本框架於 CRSCORE++ 三維度均優於基準，消融分析則將成效拆解為各元件之貢獻（詳見 §5），上述流程並以隨附開源框架實際部署於 CI/CD 與本機開發環境運作。

## 1.4 論文架構

為便於閱讀，本文章節安排如下：第二節回顧相關研究並指出現有方法之研究缺口，第三節說明所提出之審查架構與知識蒸餾訓練管線，第四節描述實驗設計、評估方法與研究問題，第五節呈現結果與分析並逐一回應各研究問題，第六節總結研究發現、限制與未來方向。

## 二、相關研究

程式碼審查為軟體開發之關鍵環節，研究指出其可在整合測試前發現約 50% 至 70% 之缺陷 [13]，且審查意見品質直接影響後續修正之成效 [14]，故自動化具高度價值。研究社群逐漸導入 LLM 作為輔助工具以提升效率並協助理解陌生領域 [2]，同時改善 Pull Request（開發者向主分支提出之合併申請）之審查延遲 [9]。另一方面，靜態分析工具（Static Analysis Tools, STAs）可於執行前檢測語法錯誤、潛在漏洞與風格問題並常整合於 CI/CD，但受限於規則僵化，相當比例之問題無法涵蓋 [5][1]，且規則更新不易 [12]、缺乏深層建議能力 [6]。

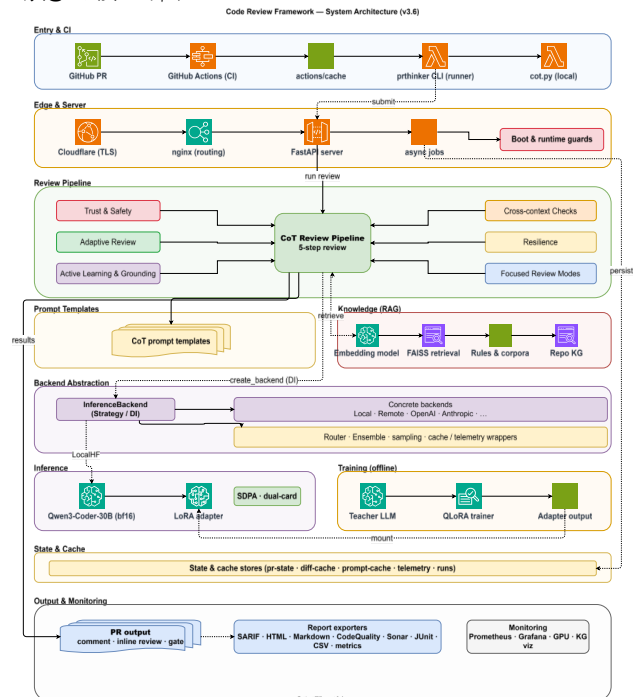
大型語言模型以 Transformer（一種以自注意力機制為核心之深度學習網路）為基礎，透過大規模預訓練與下游微調理解自然語言與程式語言，已廣泛應用於程式輔助與內容生成，惟仍面臨偏見與幻覺等問題，需搭配人工監督 [15]。評估面上，因人工標註成本高昂，LLM-as-a-Judge 以大型語言模型自動評分，其效果已證實可與人類審查者相當 [13]，並能彌補 BLEU、ROUGE 等字串重疊指標無法反映語義品質之不足 [11]，相關研究亦提出多維度指標以提升可靠性 [7][10]。為抑制幻覺，檢索增強生成（RAG）於生成前先自外部知識庫檢索相關內容並注入提示詞，可有效降低杜撰，已應用於安全建議與開發輔助系統 [8][19]。模型輕量化方面，參數高效微調（Parameter-Efficient Fine-Tuning, PEFT）僅需調整少量參數即可降低訓練成本，其中 LoRA 以低秩矩陣近似全參數更新，在極少參數下仍可逼近完整微調 [16]，QLoRA 進一步結合低位元量化，使大型模型能於有限資源下微調並維持效能 [17][21]，知識蒸餾（Knowledge Distillation）則將教師模型能力轉移至小型學生模型，於降低成本之同時維持生成品質 [22]。

綜觀上述，四條技術線各自成熟，卻分別偏向單一面向，少見整合於同一審查框架者，形成一道整合缺口：LLM 審查多採單一提示詞，難對複雜變更跨階段逐步推理，致一致性受限，自動化評分雖可行，其與人類判斷之吻合度仍需交叉驗證，RAG 與輕量化雖已成熟，卻少與多階段審查整合，使領域規範與可落地部署難以兼得。本研究即以多階段 CoT 推理、RAG 知識注入、KD/QLoRA 輕量化與雙重 Judge（自動評分加人工評分）評估四者整合，補足語義理解、跨階段一致性、領域規範與評估可信度之缺口。

## 三、LLM 程式碼審查架構設計

### 3.1 知識蒸餾與微調管線

流程先以具思維鏈之少樣本提示向教師大模型發出請求，使其於輸出審查結論之同時逐步展開中間推理，從而蒸餾出結構化且帶推理軌跡之資料而非僅有結論（教師推理軌跡係經提示隱性產生，本研究不另建自動化教師推論管線，亦不綁定特定教師模型編號）。教師輸出經清洗、轉為指令跟隨（instruction-following）格式、過濾錯誤樣本等前處理後整理為微調資料集，對學生模型（gemma-4-31B-it）進行監督式微調（Supervised Fine-Tuning, SFT），使其逼近教師之輸出分佈與推理模式。微調採低秩適應，僅更新少量低秩適配器（LoRA adapter）參數，使 30B 等級模型亦可於有限硬體高效訓練：視記憶體條件，權重可先以 4-bit 量化載入（QLoRA，表一至表三之配置採此）或直接以 bf16 載入（現行基底採此），訓練後 LoRA 適配器保持未與基座合併，以保留動態切換之彈性。



圖一、系統架構圖

### 3.2 系統整體架構

本研究提出之框架整合檢索增強生成與思維鏈多步推理，使大型語言模型於審查中得參考領域知識並逐步推理。如圖一所示，系統依職責劃分為輸入、流程編排、檢索增強、提示模板、離線訓練與推論六層，各層以明確介面相互溝通並由對應之軟體設計模式約束實作邊界，便於替換審查步驟、提示模板、檢索器或基座模型而不牽動其他層。

輸入層以 cot.py 為框架之單一入口，將模型載入與資料讀取集中於一處，便於統一管理與測試。系統啟動時即載入審查模型 gemma-4-31B-it 與訓練完成之 LoRA 權重，依序讀取待審查之程式碼，

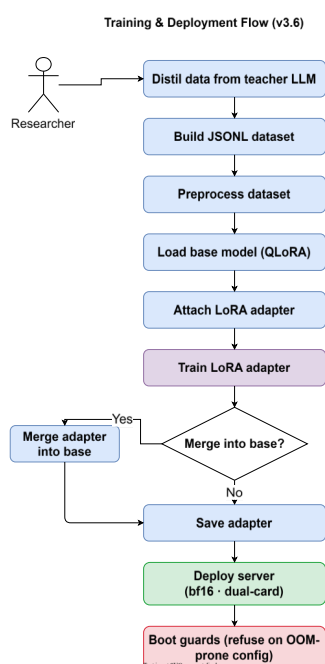
再交付後續之思維鏈管線逐步處理，使整體流程有明確且單一之進入點。

流程編排層將審查拆解為 `first_summary`、`first_code_review`、`linter`、`code_smell`、`total_summary` 五個循序步驟，分別對應變更摘要、初步審查、靜態分析、異味偵測與總結。每步皆透過 `build_global_rule_template` 注入全域規則與 RAG 檢索所得之 `top-K` 規則後，送入經 LoRA 微調之模型推理並即時寫入對應 `.md` 檔。此分步設計為本框架換取一致性之關鍵：以多階段降低單次提示詞之上下文負載，使模型專注當前子任務，並令每步產物可獨立追溯，確保長鏈推理中途失敗亦不致遺失先前產出。

檢索增強層與審查同步啟動。其直覺是：模型本身並不知道專案規範，RAG 即在每次審查前臨時把最相關的幾條專案規則塞進提示詞，讓它照規範審而不用重新訓練模型。系統由 FAISS（Facebook AI 釋出之向量索引庫，支援高速近似最近鄰搜尋）依程式碼語義檢索相關審查規則，依預設相關性閾值判斷適用性，足夠相關便注入提示詞，反之則直接進入後續審查，寧可不注入也不以弱相關規則誤導模型。

提示模板層依當前步驟構建對應之英文提示詞範本，並由 `global_rule` 模板統一包入 7 條全域審查準則（涵蓋可讀性與一致性、命名、軟體工程原則、邏輯與正確性、效能與安全、文件與測試、評分與回饋風格）作為共同前綴。唯有當 RAG 層檢索到足夠相關之規則時，才將其動態嵌入為條件式之第 8 條準則，亦即全域準則固定為 7 條靜態規則，第 8 條僅於檢索命中時出現，使外部領域知識毋須重新訓練模型即融入推理脈絡，並隨規則文件更新即時調整。

離線訓練層即 §3.1 所述之知識蒸餾與 LoRA 微調流程（如圖二所示），於有限運算資源下使較小規模之學生模型習得教師於程式碼審查任務上之推理能力，並產出供推論層載入之輕量化審查模型，此層於系統上線前完成，與線上之逐步審查解耦。



圖二、訓練流程

推論層將構建完成之提示詞送入經 LoRA 微調之 `gemma-4-31B-it` 模型執行該步驟之審查，每完成一步即判斷 CoT 流程是否結束：若尚未結束，當前結果回饋至下一輪提示詞之構建，使後續步驟延續前序之推理脈絡，形成多步漸進之審查鏈，若已全部完成，則將各步輸出注入 `total_summary` 提示詞彙整為結構化之最終審查報告。各步驟產物皆即時寫入獨立之 `.md` 檔，使每一步推理產物皆可獨立檢視與追溯。

除核心審查流程外，隨附之開源框架另實作多項工程設計並已實際部署運作：以 JudgeStep 將審查裁決映射為 GitHub Review API 之 event（`APPROVE` / `REQUEST_CHANGES` / `COMMENT`）以驅動合併閘門，並以 Strategy 介面抽象本機 Hugging Face、自架 FastAPI、OpenAI-相容與 Anthropic 四類可替換之推論後端，且已整合至 GitHub Actions CI/CD，於 Pull Request 事件對整份 PR 逐檔產生多階段審查、彙整為行內 finding 與 PR 級摘要留言並映射為合併閘門。框架另提供 Model Context Protocol（MCP）整合層與 pre-commit hook，使審查得於本機提交前或串接 MCP 時直接觸發、毋須等待 CI（開發端整合即以此二路徑為限，未另開發專屬編輯器外掛），並實作十七項研究級擴充機制（如 prompt-injection robustness、多輪對話審查與 provenance 稽核）。惟上述工程整合與十七項機制之端端品質效益不在本文評估範圍，§5 聚焦於 §3.1 之知識蒸餾與微調及 §3.2 之 RAG 與多階段 CoT 推理之驗證。

## 四、實驗設計

### 4.1 基準資料與系統配置

基準資料以 GPT-5 與 Copilot 共同生成 44 筆測試資料並經人工驗證，避免單一來源之風格偏差，使資料涵蓋語法結構、命名習慣、設計模式及錯誤型態（如潛在 bug、效能瓶頸、安全風險，以及 code smell，即功能正常但結構不良、暗示潛在維護問題之程式特徵）等多樣情境，44 筆之規模於覆蓋多維問題與控制標註成本間取得平衡。

### 4.2 評估方法

評估採 LLM-as-a-Judge 架構（如圖三所示），即請一個不參與審查的第三方模型依固定標準替每份審查報告打分，以省去大量人工標註。系統先讀取原始程式碼與審查結果，整合為一份明確定義評分標準、面向輸出格式之 Judge Prompt，輸入評審模型（如 GPT-5）後輸出一致格式之評分。其中 CRSCORE++ 要求模型先推導理想審查應涵蓋之議題集合，再分析實際 review comments 之覆蓋情形，並從 comprehensiveness（完整性）、conciseness（簡潔性）、relevance（相關性）三面向量化評分。本研究進一步提出 LLM-as-a-Judge-Our，細化為可讀性、可維護性、正確性、跨評論覆蓋性與簡潔性五項，以 1 至 100 分評分並附簡要理由。

為確保評估之客觀性，評審工作刻意交由獨立於本框架審查模型之外部大型語言模型（如 GPT-5）擔任，其用意在於：若由產生審查意見

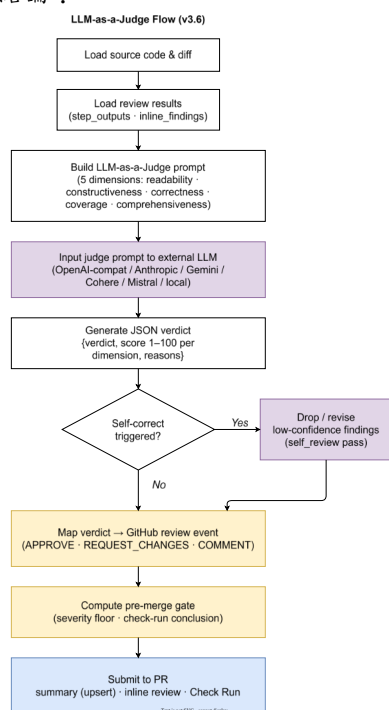


的同一模型替自己打分，難免有自利偏差，改由第三方模型評分方能得到較中立之結果。為進一步交叉驗證自動化評分之可靠度，本研究另輔以人工評分機制，由評審者就相同維度逐項評分，使整體比較得於不依賴大量人工標註之前提下，仍兼具可重現性與一致性。

#### 4.3 研究問題

本研究提出以下四個研究問題並於實驗結果進行分析：

- RQ1：結合多階段提示詞與模型微調後，整體審查品質是否優於既有基準？
- RQ2：在相同參數規模下，僅導入多階段提示詞是否即可提升審查品質？
- RQ3：多階段提示詞與模型微調二者，何者為影響審查品質之主要因素？
- RQ4：人工評分是否亦支持上述自動化評估之結論？



圖三、LLM-as-a-Judge 流程

### 五、實驗結果與分析

#### 5.1 RQ 與表格對應

為釐清研究問題與結果之對應：表一以 CRSCORE++ 之三項自動化指標回答 RQ1（完整框架之多階段提示詞加 LoRA 微調 vs 基準）與 RQ2（相同參數規模、僅多階段提示詞之效益），藉此排除模型容量差異後分離提示詞之單獨貢獻。表二拆解完整方法、微調加單一提示詞、未微調加多階段提示詞三設定，兩兩差距分別量化提示詞與微調之邊際貢獻以對應 RQ3。表三為相同三組設定之人工評分，作為對 LLM-as-a-Judge 之交叉驗證以對應 RQ4。

註：三個 Ours 欄（gemma-4-31B-it、前代基底

Qwen3-Coder-30B-A3B、小型變體 Qwen2.5-Coder-7B）均為同一 judge（Claude）、標準 CRSCORE++ 評分提示詞之評分。同一 judge 下，gemma 與前代基底於三維度相當（completeness 皆 1.00，conciseness 0.79 對 0.78，relevance 皆 0.86）。

表一、CRSCORE++ 整體評估

| 維度                | Ours (gemma-4-31B-it) | CRSCORE++ | Ours (Qwen3-Coder-30B-A3B) | Ours (Qwen2.5-Coder-7B) |
|-------------------|-----------------------|-----------|----------------------------|-------------------------|
| comprehensiveness | 1.00                  | 0.67      | 1.00                       | 0.97                    |
| conciseness       | 0.79                  | 0.57      | 0.78                       | 0.69                    |
| relevance         | 0.86                  | 0.63      | 0.86                       | 0.79                    |

#### 5.2 RQ1：整體品質與既有基準之比較

以同一 judge（Claude）、標準 CRSCORE++ 評分提示詞評分，本框架以 gemma-4-31B-it 與前代學生模型 Qwen3-Coder-30B-A3B 為基底（表一）二者於三維度相當（completeness 皆 1.00，conciseness 0.79 對 0.78，relevance 皆 0.86），顯示更換基座未損及審查品質，三個 Ours 欄於三維度均高於 CRSCORE++ 基準。

#### 5.3 RQ2：多階段提示詞之單獨效益

於相同參數量之未微調模型上套用多階段提示詞，其 comprehensiveness、relevance 與 conciseness 三維度均高於 CRSCORE++ 基準（表一，小型變體 Qwen2.5-Coder-7B 為 0.97/0.69/0.79 對基準之 0.67/0.57/0.63），顯示即使未經微調、在同等模型容量下，光是把審查流程結構化為多階段本身就能帶來實質之品質效益，可見多階段流程舉足輕重，但如下節所示，它與微調、RAG 解決的是不同問題而仍需三者協同。

#### 5.4 RQ3：提示詞與微調之邊際貢獻

由表二之消融可將整合框架之成效分解為各元件貢獻：自單一提示詞改為多階段提示詞之貢獻達 +34 分（百分制，約等於整體品質跳了三分之一個量級），導入 LoRA 微調之邊際貢獻約 +2 分。此分解並非要分高下，而是說明二者解決不同問題：多階段提示詞重塑審查之「流程結構」，使模型逐步推理而較少漏看，為單項貢獻最大之一環，LoRA 微調則在有限資源下保留並穩定教師之審查能力、使輕量學生模型得以部署，與提示詞流程互補而非競爭。

表二、提示詞設計比較（LLM 評分）

| 維度          | 微調模型<br>跟提示詞<br>設計 | 單提示詞<br>微調後模<br>型 | 基礎模<br>型跟提<br>示詞設<br>計 |
|-------------|--------------------|-------------------|------------------------|
| Readability | 92                 | 88                | 92                     |

|                       |            |    |     |
|-----------------------|------------|----|-----|
| Maintainability       | <b>95</b>  | 85 | 95  |
| Correctness           | <b>98</b>  | 82 | 98  |
| Multi-Review Coverage | <b>96</b>  | 90 | 94  |
| Conciseness           | <b>100</b> | 78 | 100 |

表三、提示詞設計比較（人工評分）

| 維度                    | 微調模型<br>跟提示詞<br>設計 | 單提示詞<br>微調後模<br>型 | 基礎模型<br>跟提示詞<br>設計 |
|-----------------------|--------------------|-------------------|--------------------|
| Readability           | 83.50              | 85.25             | <b>84.63</b>       |
| Maintainability       | <b>86.25</b>       | 79.88             | 84.88              |
| Correctness           | <b>87.75</b>       | 80.75             | 86.38              |
| Multi-Review Coverage | 86.25              | 74.13             | <b>86.38</b>       |
| Conciseness           | 84.25              | 76.25             | <b>84.75</b>       |

### 5.5 RQ4：人工評分交叉驗證

表三之人工評分與表二之 LLM 評分呈一致趨勢，於 Maintainability、Correctness、Multi-Review Coverage 與 conciseness 四維上，人工評分皆與 LLM 評分一致支持完整方法之優勢，僅 Readability 略低。此一致性一方面驗證 LLM-as-a-Judge 自動評分之可信度，另一方面亦從人工觀點佐證貢獻分解：多階段流程之優勢非孤立成立，而是與 RAG 之領域規則注入、KD/QLoRA 之輕量化互補，三者整合方便審查同時達成三性質，結論皆指向「整合」而非「單一元件」為成效來源。

## 六、結論

### 6.1 研究成果與貢獻

本研究針對將 LLM 用於程式碼審查時的三大問題：幻覺、輸出不穩定、缺乏領域規範，提出一套整合框架作為正面回應，以多階段 CoT 推理換取一致性、以 RAG 注入專案規範換取可解釋性、以知識蒸餾與 QLoRA 換取可落地之可維護性。三者整合，使審查同時具備此三性質。

經 LLM-as-a-Judge 與 CRSCORE++ 量化評估並由人工評分交叉驗證，於同一 judge 下，所提框架以 gemma-4-31B-it 與前代學生模型 Qwen3-Coder-30B-A3B 為基底，二者於三維度相當（表一）且均高於基準。消融分析則將成效分解為各元件貢獻：多階段流程之結構化設計為單項貢獻最大之一環（+34 分），LoRA 微調在有限資源下保留並穩定教師之審查能力（+2 分），RAG 提供領域規範之接地，三者互補而缺一不可。本研究的核心發現不在於某一元件勝出，而在於唯

有三者整合方能同時改善三性質、回應前述三大問題。

### 6.2 研究限制與未來工作

本研究仍有若干限制與後續發展方向，於此一併說明，以利後續研究接續推進。資料集方面，目前以 44 筆、以 Python 為主之資料驗證，規模仍屬有限，所涵蓋之語言與專案型態尚不夠廣，未來可擴展至多種程式語言、多框架與真實開源專案，以檢驗框架於更大樣本與更異質情境下之穩定性。模型能力方面，本研究主要針對單一學生模型進行微調，未來可進一步探索多模型協作（multi-agent）之可能，使各模型分別負責特定面向（如安全、效能、風格），再彙整為更全面之審查意見。評估範圍方面，§3.2 所述之框架雖已部署於 GitHub Actions CI/CD、MCP 與 pre-commit 等路徑實際運作，惟其多後端推論之品質、成本與延遲，反饋語料累積所帶來之效益，以及十七項擴充機制之端到端效益均尚未量化評估，本文亦僅以各維度之平均分數呈現結果，未進一步進行統計顯著性檢定，消融分析也尚未單獨分離 RAG 注入之貢獻（結果皆於檢索層啟用之配置下取得）。凡此種種，均留待後續研究逐項驗證與補足。

### 參考文獻

- [1] I. Jaoua, O. B. Sghaier, and H. Sahraoui, "Combining large language models with static analyzers for code review generation," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025: IEEE, pp. 174–186.
- [2] F. S. Aðalsteinsson, B. B. Magnússon, M. Milicevic, A. N. Davidsson, and C.-H. Cheng, "Rethinking code review workflows with LLM assistance: An empirical study," in *2025 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2025, pp. 488–497.
- [3] P. J. S. Xuan and Y. Lee, "Automated Code Review and Bug Detection," in *2024 19th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, 2024: IEEE, pp. 1–6.
- [4] A. Frömmgen et al., "Resolving code review comments with machine learning," in *Proceedings of the 46th international conference on software engineering: software engineering in practice*, 2024, pp. 204–215.
- [5] M. Vijayvergiya et al., "AI-assisted assessment of coding practices in modern code review," in *Proceedings of the 1st ACM international conference on AI-powered software*, 2024, pp. 85–93.
- [6] Z. Rasheed et al., "AI-powered code review with LLMs: Early results," *arXiv preprint arXiv:2404.18496*, 2024.
- [7] A. Naik, M. Alenius, D. Fried, and C. Rose, "CRScore: Grounding automated evaluation of code review comments in code claims and smells," in *Proceedings of the 2025 Conference of the Nations of*

*the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2025, pp. 9049–9076.

[8] T. Sharanarathi and S. Polineni, "Multi-Agent LLM Collaboration for Adaptive Code Review, Debugging, and Security Analysis," in *2025 International Conference on Mechatronics, Robotics, and Artificial Intelligence (MRAI)*, 2025: IEEE, pp. 541–546.

[9] C. Adapa, S. S. Avulamanda, A. Anjana, and A. Victor, "AI-powered code review assistant for streamlining pull request merging," presented at the *2024 IEEE International Conference for Women in Innovation, Technology & Entrepreneurship (ICWITE)*, 2024.

[10] M. N. Kapadnis, A. Naik, and C. Rose, "CRScore++: Reinforcement learning with verifiable tool and AI feedback for code review," *arXiv preprint arXiv:2506.00296*, 2025.

[11] G. Crupi, R. Tufano, A. Velasco, A. Mastropaolo, D. Poshyanyk, and G. Bavota, "On the Effectiveness of LLM-as-a-judge for Code Generation and Summarization," *IEEE Transactions on Software Engineering*, 2025.

[12] L. Yang and Y. Chen, "Integrating RAG and LLM for Automated Code Review in Practice," in *2025 10th International Conference on Computer and Information Processing Technology (ISCIPIT)*, 2025: IEEE, pp. 591–596.

[13] T. Kakimoto, H. Inayoshi, H. Uwano, and A. Monden, "Evaluating the Source Code Review Performance of LLM-based AI Chatbots," in *2025 IEEE/ACIS 29th International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)*, 2025: IEEE, pp. 593–600.

[14] O. B. Sghaier, M. Weyssow, and H. Sahraoui, "Harnessing large language models for curated code reviews," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025: IEEE, pp. 187–198.

[15] D. O. Salem, Y. Alahmed, R. Fnich, M. Mazroub, and M. Fnich, "AI-Driven Continuous Integration: Automating Code Review and Deployment with LLMs," in *2025 10th International Conference on Fog and Mobile Edge Computing (FMEC)*, 2025: IEEE, pp. 268–274.

[16] J. Lu, L. Yu, X. Li, L. Yang, and C. Zuo, "Llama-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning," in *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, 2023: IEEE, pp. 647–658.

[17] M. A. Haider, A. B. Mostofa, S. S. B. Mosaddek, A. Iqbal, and T. Ahmed, "Prompting and fine-tuning large language models for automated code review comment generation," *arXiv preprint arXiv:2411.10129*, 2024.

[18] C. Liu, H. Y. Lin, and P. Thongtanunam, "Too

noisy to learn: Enhancing data quality for code review comment generation," in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, 2025: IEEE, pp. 236–248.

[19] Y. Chen, "AutoReview: An LLM-based Multi-Agent System for Security Issue-Oriented Code Review," in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025, pp. 1022–1024.

[20] Y. Zhang, Y. Zhang, Z. Sun, Y. Jiang, and H. Liu, "Laura: Enhancing code review generation with context-enriched retrieval-augmented LLM," in *2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2025: IEEE, pp. 2983–2995.

[21] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *Advances in neural information processing systems*, vol. 36, pp. 10088–10115, 2023.

[22] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.